

EXPLORING MLRSNET: A GAME – CHANGER IN REMOTE SENSING

A. Disha Ghosh¹

MSc. in Data Science
Technical University of Braunschweig

B. Nizaanth Raja²

MSc. in Computational Science
Technical University of Braunschweig

ABSTRACT

To enhance semantic scene understanding in remote sensing, multi-label annotation is crucial. Deep learning models require large-scale annotated data, but most existing datasets use single labels, which cannot fully describe the complexity of remote sensing images. In this paper, we introduce MLRSNet, a multi-label high-resolution remote sensing dataset for deep learning-based scene understanding. MLRSNet contains 109,161 samples across 46 scene categories, with each image labeled using one or more of 60 predefined labels. We use the EfficientNet base model to establish benchmarks for multi-label image classification and compare its performance to other deep learning methods from the original MLRSNet paper. We introduce two novel models: one that employs learning rate decay using step decay, and another that uses performance-based scheduling. Our results show that MLRSNet is a valuable benchmark for multi-label image research.

Index Terms— Multilabel Classification, Remote Sensing, MLRSNet dataset, Deep Learning, Learning Rate Decay.

1. INTRODUCTION

The rapid advancement in remote sensing imagery has significantly driven interest in leveraging deep learning architectures for sophisticated image analysis. Notable models such as InceptionV3 (Szegedy et al., 2016), VGGNet16 and VGGNet19 (Simonyan & Zisserman, 2015), ResNet50 and ResNet101 (He et al., 2016), and DenseNet121, DenseNet169, and DenseNet201 (Huang et al., 2017) have each introduced innovative approaches to enhancing image classification tasks. InceptionV3 is distinguished by its use of multiple convolutional filter sizes within a single layer, allowing for improved feature extraction at various scales. VGGNet architectures, known for their deep and uniform layer structure with small 3x3 filters, have set a new standard for depth and feature richness. ResNet models introduced the concept of residual learning, which facilitates the training of much deeper networks by addressing the vanishing gradient problem. DenseNet architectures, on the other hand, leverage dense

connectivity between layers to promote feature reuse and gradient flow, further enhancing performance and efficiency.

Despite their advancements, these architectures face specific challenges. InceptionV3's complexity results in high computational and memory demands, which can be a barrier in resource-constrained environments. VGGNet16 and VGGNet19, while effective, are susceptible to overfitting due to their large number of parameters, especially when dealing with limited datasets. ResNet50 and ResNet101, although successful in mitigating vanishing gradients, suffer from increased computational load and extended training times, making them less efficient in certain applications. DenseNet models, while offering improved gradient flow and feature reuse, encounter issues related to substantial memory consumption and prolonged training durations. These challenges highlight the need for continued innovation and optimization in deep learning methods to address the complexities of remote sensing image analysis (Toth & Józków, 2016).

To overcome the aforementioned issues and improve the understanding of ground objects, in this report, we propose the use of EfficientNet as a more effective deep learning architecture for multi-label remote sensing image classification. EfficientNet is designed to address the limitations of traditional models by optimizing both accuracy and computational efficiency. It achieves this through a compound scaling method, which uniformly scales network depth, width, and resolution in a balanced manner, ensuring improved performance without excessive computational overhead. Unlike traditional models like InceptionV3, VGGNet, ResNet, and DenseNet, which require larger memory and computational resources, EfficientNet delivers state-of-the-art performance while significantly reducing the number of parameters and FLOPs (Floating Point Operations).

In this study, we specifically evaluate the use of multi-label classification within the EfficientNet framework for remote sensing image analysis. Remote sensing data often contain intricate relationships between various land cover classes,

such as roads and vehicles or water bodies and vegetation. By applying multi-label classification, EfficientNet is able to capture these complex label correlations more effectively than single-label approaches. Our experiments demonstrate that EfficientNet achieves superior performance in terms of accuracy and computational efficiency compared to traditional architectures. This highlights its potential for large-scale remote sensing applications, offering a robust solution for overcoming the challenges of scene understanding and classification in high-resolution imagery.

2. MLRSNET DATASET

MLRSNet is a large-scale dataset comprising 109,161 labeled RGB images sourced from various locations around the world, providing a diverse representation of global landscapes. These images are grouped into 46 broad categories, such as *airplane*, *airport*, *bareland*, *baseball diamond*, *basketball court*, *beach*, *bridge*, *chaparral*, *cloud*, *commercial area*, *dense residential area*, *desert*, *eroded farmland*, *farmland*, *forest*, *freeway*, *golf course*, *ground track field*, *harbor & port*, *industrial area*, *intersection*, *island*, *lake*, *meadow*, *mobile home park*, *mountain*, *overpass*, *park*, *parking lot*, *parkway*, *railway*, *railway station*, *river*, *roundabout*, *shipping yard*, *snowberg*, *sparse residential area*, *stadium*, *storage tank*, *swimming pool*, *tennis court*, *terrace*, *transmission tower*, *vegetable greenhouse*, *wetland*, and *wind turbine*. The distribution of sample images varies across categories, with each class containing between 1,500 and 3,000 images, as shown in Fig. 1.

In addition to these broad categories, each image is further annotated with multiple labels drawn from a predefined set of 60 class labels. These annotations capture the complex relationships between different land cover elements, with each image assigned between 1 and 13 labels. The distribution of these images per label are shown in Fig.3. This multi-label annotation allows for a more detailed and nuanced understanding of the scenes depicted in the images. Furthermore, this multi-label structure provides rich data for advanced scene understanding tasks in remote sensing, helping to uncover intricate correlations between objects such as "roads" and "vehicles" or "water" and "vegetation." Fig. 2 showcases several sample images

along with their corresponding multi-label annotations, demonstrating the dataset's diversity and complexity.

3. EFFICIENTNET ARCHITECTURE

EfficientNet introduces a novel approach to scaling neural networks by addressing three critical dimensions in a neural network's architecture: depth, width, and resolution. Depth refers to the number of layers, specifically convolutional layers or blocks, in the network. Increasing the depth allows the network to extract more complex and hierarchical features from the input data, but it also introduces challenges like vanishing gradients and overfitting, which make training deeper networks difficult. Width, on the other hand, refers to the number of channels (also known as filters or feature maps) in each convolutional layer. A wider network is capable of capturing more fine-grained features in the data, but scaling width comes with the risk of increased computational redundancy and a rise in resource consumption. Resolution refers to the input image size processed by the network. Higher resolution images contain more detailed information, enabling the network to learn finer visual patterns, but they also demand significantly more memory and processing power, complicating scalability. The key concepts of depth, width and resolution are shown in Fig.4.

EfficientNet tackles these challenges by introducing a **Compound Scaling** [7] that scales depth, width, and resolution in a balanced manner. Instead of increasing these dimensions independently—which could result in inefficiencies—EfficientNet employs scaling factors for each dimension: depth is scaled by $\alpha\phi$, width by $\beta\phi$, and resolution by $\gamma\phi$, where ϕ is a user-defined coefficient that uniformly scales the model. The design rule $\alpha \times \beta^2 \times \gamma^2 \approx 2$ ensures that the model grows in a balanced way, optimizing performance while maintaining computational efficiency. To demonstrate this scaling, the baseline model, EfficientNet-B0, was developed using Neural Architecture Search (NAS) [2], which optimizes both accuracy and FLOPS (Floating Point Operations). The core of EfficientNet's architecture is the **Mobile Inverted Bottleneck (MBConv)** [3], which was first introduced in MobileNetV2 to reduce the number of parameters while maintaining strong performance. Additionally, EfficientNet

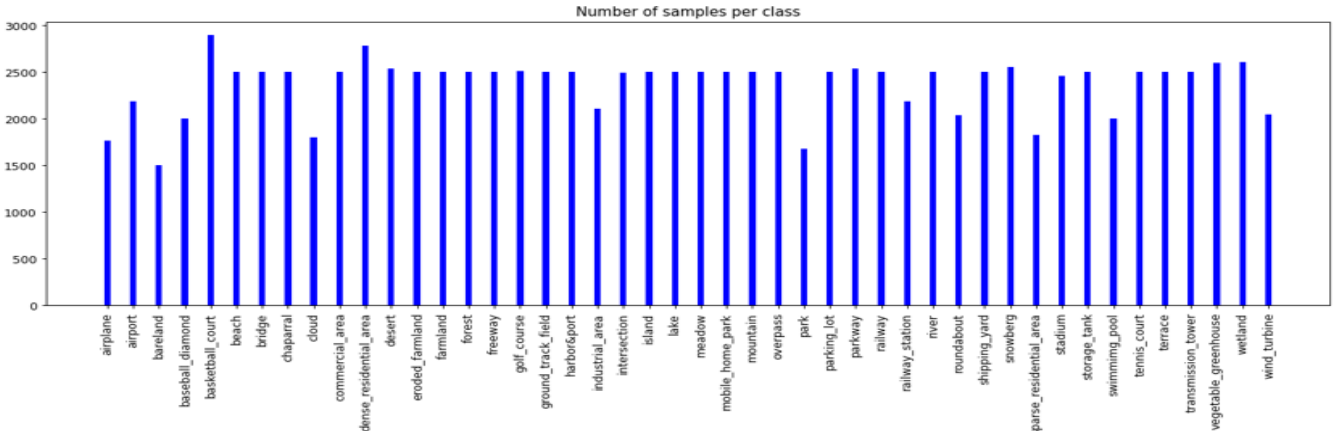


Fig. 1. Distribution of Number of samples per category*



Fig.2. Labels and Categories for each image*

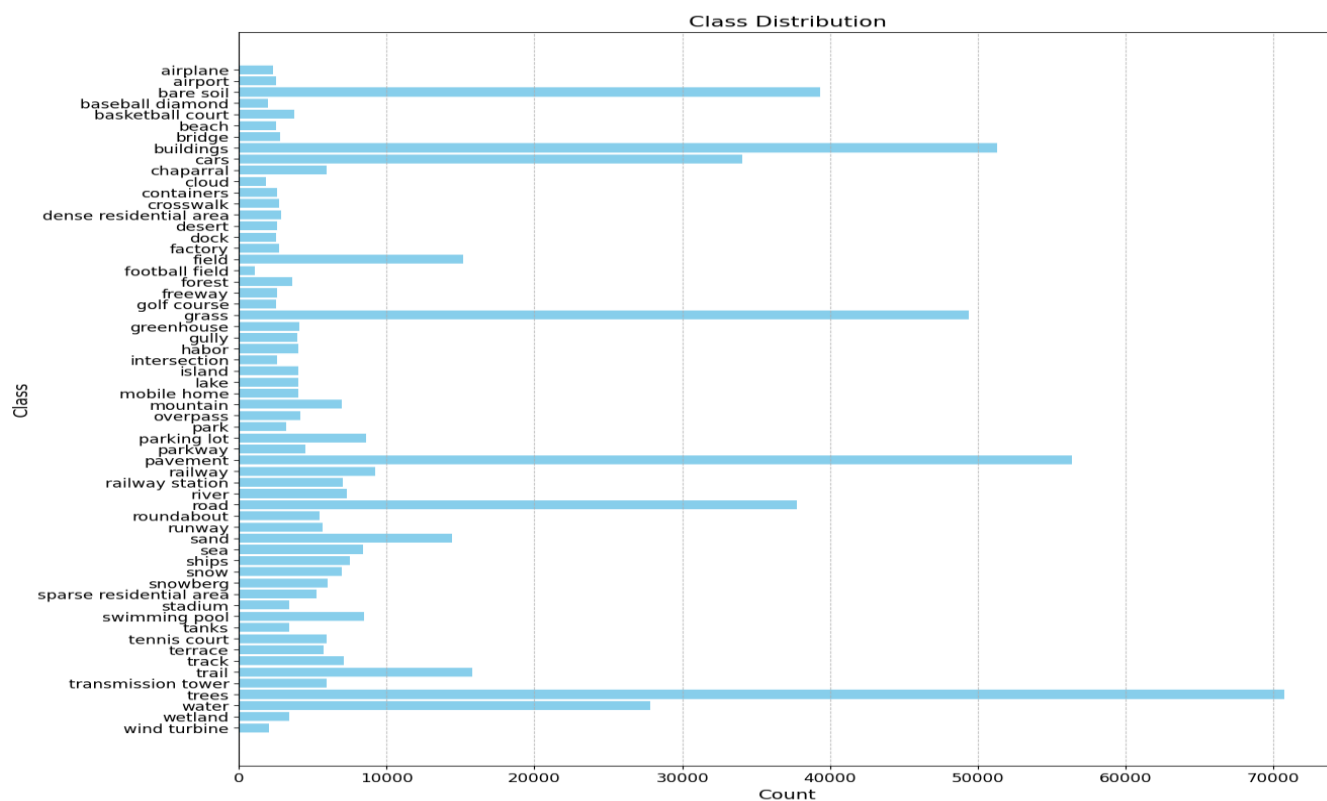


Fig.3. Distribution of Images per Label*

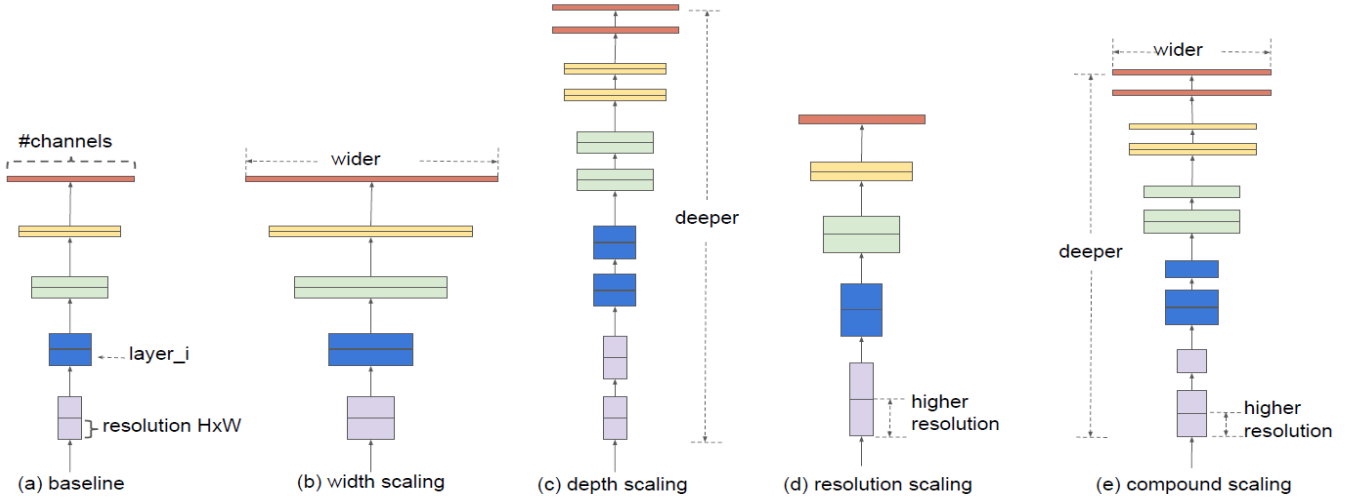


Fig.4. **Model Scaling Visualisation.** (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio. [1]

integrates **squeeze-and-excitation optimization** [4], which improves the network's ability to focus on the most relevant features by recalibrating channel-wise feature responses. Together, these architectural innovations allow EfficientNet to deliver state-of-the-art accuracy while requiring fewer parameters and lower computational resources compared to traditional deep learning models.

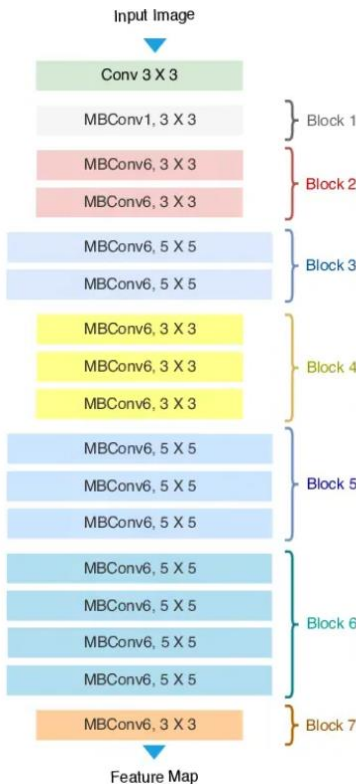


Fig.5. Compact EfficientNet Architecture [5]

4. EXPERIMENTAL SETUP

4.1. Data Split Ratios

To evaluate the classification performance of various Convolutional Neural Networks (CNNs) on the MLRSNet dataset, three different training-validation-testing splits are employed, as outlined in the original MLRSNet paper. The first split uses 20% of the dataset for training, 10% for validation, and 70% for testing. The second split increases the training portion to 30%, maintains 10% for validation, and reserves 60% for testing. The third split expands the training set to 40%, with 10% for validation, and 50% for testing. These splits, taken from the MLRSNet paper, provide a detailed evaluation of CNN performance with varying amounts of training data. Importantly, for all three splits, the models are initialized with pre-trained weights from ImageNet, following the methodology from the MLRSNet study [6].

In addition to these pre-trained setups, a **scratch model** is also tested. In this configuration, the model is trained without using pre-trained weights, meaning it starts from random initialization. In the scratch model, **70%** of the dataset is allocated for training, **10%** for validation, and **20%** for testing. This setup helps analyze the model's ability to learn complex features entirely from the MLRSNet dataset without the benefit of previously learned knowledge from other datasets like ImageNet. A concise form is shown in Table 1.

For all models, regardless of whether they are pre-trained or trained from scratch, **binary cross-entropy** is used as the loss function. Since the task involves multi-label

classification, binary cross-entropy is ideal for measuring how well the predicted probabilities match the true labels (either 0 or 1) for each class. The function calculates individual errors for each class, averages them, and updates the model accordingly to minimize overall prediction error. This consistent use of binary cross-entropy ensures a uniform approach for comparing the performance of different CNN architectures across all data splits and training methods.

The formula for binary cross – entropy is as follows:

$$L = \frac{1}{mq} \sum_{i=1}^m \sum_{j=1}^q (t_i^j \log p_i^j + (1 - t_i^j) \log (1 - p_i^j))$$

Where, $t_i^j \in (0,1)$ denotes the ground – truth label for training image X_i . p_i^j is the output of the sigmoid layer. m is the number of training images and q is the number of the classes in total.

SETUP	TRAINING	VALIDATION	TESTING	PRE – TRAINED WEIGHTS (IMAGENET)
SPLIT 1	20%	10%	70%	YES
SPLIT 2	30%	10%	60%	YES
SPLIT 3	40%	10%	50%	YES
SCRATCH	70%	10%	20%	NO

Table 1. Split ratios used in the project

Package	Epochs	Batch Size	Optimizer	Learning Rate
Keras	10	32	Adam	0.01

Table 2. Hyper parameters used for the split ratios

4.2. Evaluation Protocols

To quantitatively evaluate the classification results, two widely used evaluation metrics are computed: mean average precision (mAP) and F1 score. The F1 score is a comprehensive metric that balances precision and recall, offering a robust measure of classification performance for each model. The F1 score is calculated as follows:

$$F_1 = \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \times 2$$

Here, **precision** is defined as the ratio of correctly predicted labels to the total predicted labels for a sample:

$$\text{Precision} = \frac{|L_c \cap L_r|}{|L_c|}$$

where L_c represents the predicted label vector and L_r is the ground-truth label vector for a sample. Similarly, **recall** is the ratio of correctly predicted labels to the total ground-truth labels:

$$\text{Recall} = \frac{|L_c \cap L_r|}{|L_r|}$$

A higher F1 score indicates better classification performance. It's important to note that precision and recall are calculated separately for each sample and then averaged across all samples to compute the final F1 score.

For **mean average precision (mAP)**, the precision for each label is computed, and the final mAP score is derived by averaging these precision values across all labels. This metric helps in understanding how well the model performs in predicting multiple labels across the entire dataset. Both metrics provide a balanced and comprehensive evaluation of the classification performance of different models.

For the Scratch model, in addition to the aforementioned metrics, we all also incorporated the following evaluation metrics.

Binary Accuracy: The proportion of correct predictions (both true positives and true negatives) among the total number of predictions.

$$\text{Binary Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

ROC AUC: Measures the area under the ROC curve, which plots the true positive rate versus the false positive rate across different threshold settings.

$$\text{ROC AUC} = \int_0^1 \text{TPR}(x) d\text{FPR}(x)$$

Where, TPR(x) = True Positivity Rate (Sensitivity)

FPR(x) = False Positive Rate (Specificity)

Hamming Loss: The Fraction of labels that are incorrectly predicted in multi – label classification.

$$\text{Hamming Loss} = \frac{1}{N} \sum_{i=1}^N \frac{1}{L} \sum_{j=1}^L (P_{i,j} \neq Q_{i,j})$$

Where, N = number of samples,

L = number of labels,

$P_{i,j}$ = predicted label for the i-th sample and j-th label,

$Q_{i,j}$ = true label for the i-th sample and j-th label

4.3. Learning Rate Decay

Step decay is a learning rate schedule technique where the learning rate is reduced by a fixed factor after a predefined number of training steps or epochs. This method is used to gradually lower the learning rate as the model progresses through training, enabling faster convergence during the early stages and more fine-tuned adjustments in the later stages. The goal of step decay is to achieve better optimization by adapting the learning rate over time, preventing the model from overshooting optimal values early in training and improving the model's performance during fine-tuning. The formula for step decay is:

$$\alpha_t = \alpha_0 d^{\frac{t}{s}}$$

Where, α_0 is the initial learning rate, t is the iteration step, $d < 1$: is the decay rate and s are the drop rate.

In our implementation, initially, the learning rate is set at **0.005**, which is sufficiently large to allow the model to converge rapidly in the early stages of training. To gradually fine-tune the model's parameters, we decay the learning rate by **10% every 5 epochs**. This means that after every 5 epochs, the learning rate is multiplied by 0.9, reducing it incrementally to ensure more controlled adjustments as training progresses.

We continue this process over the course of 200 epochs, allowing the learning rate to start high and decay progressively. This step decay schedule is essential for achieving a balance between fast convergence at the beginning and fine-tuned weight optimization in the later stages. The early large learning rate helps the model make significant progress quickly, while the subsequent reductions help prevent overshooting and improve overall performance.

4.4. Performance Scheduling

Performance scheduling in neural network training involves configuring and adjusting hyperparameters to optimize model performance. Key elements include setting the batch size (number of samples processed per iteration), learning rate (step size for weight updates), and number of epochs (complete passes through the dataset). A common technique is learning rate scheduling, where the learning rate is adjusted based on training metrics. For instance, if validation loss or accuracy stops improving, the learning rate may be reduced by a fixed factor to refine the model's learning. This approach, known as "Reduce Learning Rate on Plateau," helps maintain efficient training and improves model performance by adapting learning rates when progress stalls.

In our project, we implemented performance scheduling with specific hyperparameters to optimize training. We used a batch size of 32, balancing between computational efficiency and model stability. The initial learning rate was set to 0.005 to facilitate rapid convergence initially while allowing for effective adjustments as training progressed. We trained the model for 200 epochs to ensure adequate learning and refinement of the model's weights. This configuration aims to achieve a balance between fast learning and thorough optimization, ultimately improving the model's performance and generalization capabilities.

One thing to note here, implemented early stopping as part of our performance scheduling strategy to monitor validation loss. If the validation loss did not improve for 20 consecutive epochs (patience set to 20), the training process was halted. In the case of step decay, the validation loss curve indicated significant potential for further improvement, as it did not flatten out. To address this, we opted for performance scheduling, allowing us to reduce the learning rate whenever the validation loss plateaued. This approach ultimately resulted in a more stable, flattened validation loss curve.

5. RESULTS AND VISUALIZATIONS

CNNs	20%	30%	40%
MLRSNet-InceptionV3	81.50	82.33	84.84
MLRSNet-VGGNet16	67.88	72.66	75.39
MLRSNet-VGGNet19	66.12	69.53	73.60
MLRSNet-ResNet50	82.65	84.28	86.01
MLRSNet-ResNet101	83.26	84.19	85.72
MLRSNet-DenseNet121	75.96	77.99	80.25
MLRSNet-DenseNet169	82.16	86.42	87.35
MLRSNet-DenseNet201	87.25	87.84	88.77
MLRSNet-EfficientNetB0	93.10	94.10	94.58

Table 3. Mean Average Precision (mAP): Comparison of our model with the other eight fine - tuned model

CNNS	20%	30%	40%
MLRSNet-InceptionV3	0.7746	0.8016	0.8146
MLRSNet-VGGNet16	0.5743	0.6534	0.6855
MLRSNet-VGGNet19	0.5677	0.6120	0.6329
MLRSNet-ResNet50	0.7530	0.8176	0.8353
MLRSNet-ResNet101	0.7618	0.7703	0.8226
MLRSNet-DenseNet121	0.7154	0.7389	0.7571
MLRSNet-DenseNet169	0.8138	0.8408	0.8521
MLRSNet-DenseNet201	0.8381	0.8414	0.8538
MLRSNet-EfficientNetB0	0.8355	0.8492	0.8539

Table 4. F₁ Score (Samples): Comparison of our model with the other eight fine - tuned model mentioned in the paper

Metrics	Performance Scheduling	Step Decay
F1 Score (Samples)	0.8568	0.8395
Binary Accuracy	97.58	97.15
ROC AUC Curve	99.22	98.93
Mean Average Precision	94.61	93.53
Hamming Loss	2.41	2.84

Table 5. Evaluation metrics results for the scratch models

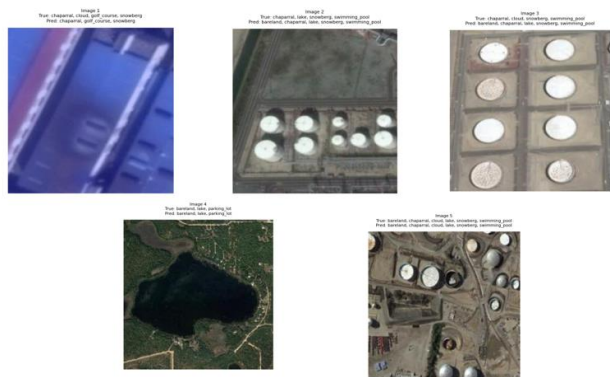


Fig.6. Visualisations for split ratio 20-10-70*

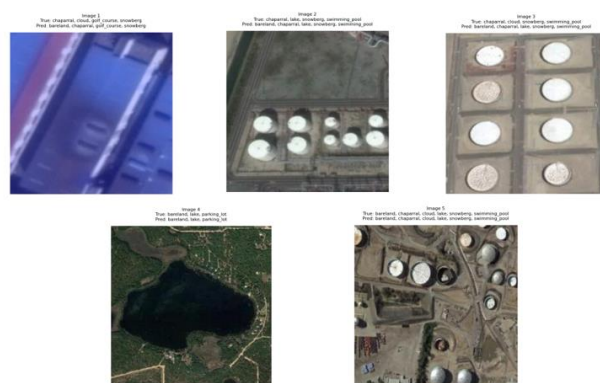


Fig.7. Visualisation for split ratio 30-10-60*

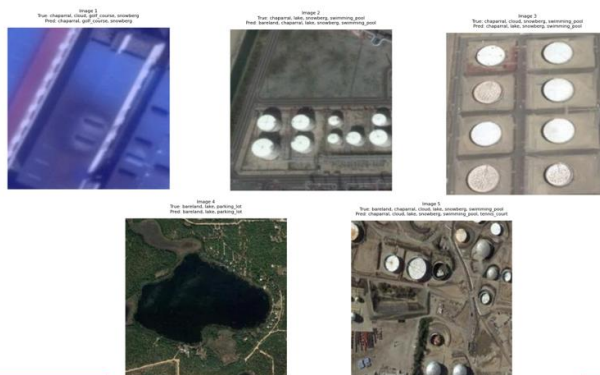


Fig.8. Visualisation for split ratio 40-10-50*



Fig.9. Visualisation for Performance Scheduling*



Fig.10. Visualisation for Step Decay*

6. REFERENCES

* For clearer images and further details, please refer to the .png files and the corresponding code available on GitHub.

** The contents provided here have been reframed using AI to enhance sentence structure and clarity.

For the training runs, and graphs on validation loss over epochs and in - depth information, please click on the references mentioned here.

1. [Wandb Report to EfficientNet Base Model B0 for the Split Ratio: 20-10-70.](#)
2. [Wandb Report to EfficientNet Base Model B0 for the Split Ratio: 30-10-60.](#)
3. [Wandb Report to EfficientNet Base Model B0 for the Split Ratio: 40-10-50.](#)
4. [Comparison between Performance Scaling and Step Decay - WandB Report](#)
5. [WandB Report to the EfficientNet Base Model B0 for the Step Decay.](#)
6. [WandB Report for the EfficientNet Base Model B0 for the Performance Scaling.](#)

For the source code and the output images, please refer the GitHub link: https://github.com/Deep-Learning-IGP-TUBS-SoSe2024/Group_01/tree/main/Final_Project/03-Code

[1]. Tan, M. and Le, Q.V. (2020) *EfficientNet: Rethinking model scaling for Convolutional Neural Networks*, *arXiv.org*. Available at: <https://arxiv.org/abs/1905.11946> (Accessed: 16 September 2024).

[2]. Tan, M. et al. (2019) *MnasNet: Platform-aware neural architecture search for mobile*, *arXiv.org*. Available at: <https://arxiv.org/abs/1807.11626> (Accessed: 16 September 2024).

[3]. Sandler, M. et al. (2019) *MobileNetV2: Inverted residuals and linear bottlenecks*, *arXiv.org*. Available at:

<https://arxiv.org/abs/1801.04381> (Accessed: 16 September 2024).

[4]. Hu, J. et al. (2019a) *Squeeze-and-excitation networks*, *arXiv.org*. Available at: <https://arxiv.org/abs/1709.01507> (Accessed: 16 September 2024).

[5]. Ahmed, T. and Sabab, N.H.N. (2021) *Classification and understanding of cloud structures via satellite images with EfficientUNet*, *arXiv.org*. Available at: <https://arxiv.org/abs/2009.12931> (Accessed: 16 September 2024).

[6]. Author links open overlay panel Xiaoman Qi a 1 et al. (2020) *MLRSNet: A multi-label high spatial resolution remote sensing dataset for semantic scene understanding*, *ISPRS Journal of Photogrammetry and Remote Sensing*. Available at: <https://www.sciencedirect.com/science/article/abs/pii/S0924271620302677> (Accessed: 16 September 2024).

[7]. Thomas, A. (2023) *EfficientNet implementation from scratch in pytorch: A step-by-step guide*, *Medium*. Available at: <https://medium.com/@aniketthomas27/efficientnet-implementation-from-scratch-in-pytorch-a-step-by-step-guide-a7bb96f2bdaa> (Accessed: 16 September 2024).

[8]. Tensorflow *TPU/models/official/efficientnet at master · TensorFlow/TPU*, *GitHub*. Available at: <https://github.com/tensorflow/tpu/tree/master/models/official/efficientnet> (Accessed: 16 September 2024).

[9]. Pytorch *Vision/torchvision/models/efficientnet.py at main · Pytorch/Vision*, *GitHub*. Available at: <https://github.com/pytorch/vision/blob/main/torchvision/models/efficientnet.py> (Accessed: 16 September 2024).

[10]. Aladdinpersson Machine-learning-collection/ml/pytorch/cnn_architectures/pytorch_efficientnet.py at master · Aladdinpersson/machine-learning-collection, *GitHub*. Available at: https://github.com/aladdinpersson/Machine-Learning-Collection/blob/master/ML/Pytorch/CNN_architectures/pytorch_efficientnet.py (Accessed: 16 September 2024).

[11]. Stojmichev, M., Koccev, D., & Džeroski, S. (2023, January 16). *Deep network architectures as feature extractors for multi-label classification of Remote Sensing Images*. *MDPI*. <https://www.mdpi.com/2072-4292/15/2/538>